Health Transformation Review — AI & RAG System Guide

For developers and operators who need to understand, maintain, or extend the HTR AI Analyst.

System Overview

The HTR AI Analyst is a Retrieval-Augmented Generation (RAG) system. It combines:

- A **vector knowledge base** built from HTR content (PDFs + Sanity CMS)
- A **Groq LLM** (`llama-3.3-70b-versatile`) that generates responses
- **OpenAI embeddings** (`text-embedding-3-small`) that power semantic search
- **LlamaIndex** as the RAG orchestration framework
- **Supabase pgvector** as the production vector store

Architecture

```
USER QUESTION
     │
     ▼
Frontend /api/chat (Next.js proxy)
     │ forwards with Authorization header
     ▼
Backend POST /api/chat (FastAPI)
     │
     ├── 1. JWT verification (Supabase)
     │       └── Fetches user role from user_roles table
     │
     ├── 2. Role check (subscriber or higher required)
     │
     ├── 3. Rebuild ChatMemoryBuffer from conversation history
     │
     ├── 4. Select system prompt based on user role
     │
     ├── 5. Create chat engine from VectorStoreIndex
     │       └── chat_mode="context" (RAG-enhanced)
     │
     ├── 6. Execute astream_chat(message)
     │       │
     │       ├── Embed question → text-embedding-3-small
     │       ├── Similarity search → Supabase pgvector (rag_documents table)
     │       ├── Retrieve top-N relevant chunks
     │       └── Prompt Groq LLM with context + question + history
     │
     └── 7. Stream tokens back via StreamingResponse
```

Knowledge Base

Content Sources

Two source types are indexed:

1. PDFs (`backend/data/` directory)

Any `.pdf` file placed in `backend/data/` is automatically detected and ingested. Current files:

- `Act 167 CE_St. Johnsbury Presentation_v04.pdf` — Vermont Act 167 Community Engagement presentation
- `Health Economics.pdf` — Health economics reference material

- `Wyman_Report.pdf` — Wyman research report

PDF pages are split into individual documents by `SimpleDirectoryReader`. Each page gets `source: "pdf"` and `pillar: "General"` metadata.

2. Sanity CMS content

Eight content types are queried from Sanity via GROQ HTTP API:

Content Type	Fields Extracted
`policyAnalysis`	title, pillar, summary, body (as plain text)
`post`	title, body (as plain text)
`academyModule`	title, pillar, summary, learningObjectives, body (as plain text)
`caseStudy`	title, pillar, summary, body (as plain text)
`definition`	term, description, pillars
`analystNote`	title, pillar, body (as plain text)
`webinar`	title, pillar, description
`report`	title, pillar, abstract

Only documents with a defined `slug.current` are included (definitions and analyst notes are included regardless of slug).

Documents shorter than 20 characters are discarded. Documents are truncated to 8,000 characters each.

Document Metadata

Each document in the vector store carries:

```
{
    "source": "pdf" | "policyAnalysis" | "post" | "academyModule" | ...,
    "doc_id": "_id from Sanity or filename",
    "title": "document title",
    "pillar": "Policy" | "Economics" | "Technology" | "Clinical" | "Equity" |
"General"
}
```

Vector Store

- **Production**: Supabase pgvector, table `rag_documents`
- **Fallback**: Local JSON files in `backend/storage/`
- **Embedding dimensions**: 1536 (text-embedding-3-small)
- **Connection**: Requires `SUPABASE_DB_URL` (PostgreSQL connection string with pooler)

```
PGVectorStore.from_params(
    connection_string=SUPABASE_DB_URL,
    async_connection_string=async_db_url,  # postgresql+asyncpg://...
    table_name="rag_documents",
    embed_dim=1536,
)
```

Index Lifecycle

Startup sequence

When the FastAPI server starts, `startup()` runs:

1. `load_index()` — tries to load existing index from Supabase pgvector, then local storage fallback
2. If found: index is ready immediately (fast start, no re-embedding)
3. If not found: `build_index()` runs — embeds all documents and stores in pgvector

Building the index (`build_index()`)

1. Load all PDFs from `backend/data/`
2. Fetch all 8 content types from Sanity via GROQ HTTP API (with retry logic: 3 attempts, exponential backoff for rate limits)
3. Combine all documents
4. Embed via OpenAI `text-embedding-3-small`
5. Store in Supabase pgvector (or local JSON fallback)

Build time depends on content volume. Typical: 1–3 minutes for a full rebuild.

Rebuilding the index

Trigger via the `/api/ingest` endpoint:

```
# Production
curl -X POST https://your-backend.railway.app/api/ingest \
  -H "Authorization: Bearer $INGEST_SECRET"

# Dev (no secret required if INGEST_SECRET not set)
curl -X POST http://localhost:8000/api/ingest
```

The rebuild runs in a background asyncio task. The existing index remains active and serves requests during the rebuild. The `_index` global is swapped atomically under `_index_lock` when the new index is ready.

When to rebuild:

- After publishing significant new content in Sanity
- After adding new PDFs to `backend/data/`
- After modifying existing content that the AI should reflect

---

LLM Configuration

Primary LLM

- **Provider**: Groq
- **Model**: `llama-3.3-70b-versatile` (configurable via `GROQ_MODEL` env var)
- **Purpose**: Chat responses and suggestion generation

Configured via LlamaIndex `Settings`:

```
Settings.llm = GroqLLM(model=GROQ_MODEL, api_key=GROQ_API_KEY)
```

Embedding Model

- **Provider**: OpenAI
- **Model**: `text-embedding-3-small` (hardcoded, not configurable)
- **Dimensions**: 1536
- **Purpose**: Encoding documents and queries for vector similarity search

```
Settings.embed_model = OpenAIEmbedding(model="text-embedding-3-small",
api_key=OPENAI_API_KEY)
```

Temperature

Default: 0.7. Clients can override via `temperature` field in the request (clamped to 0.0–1.0).

System Prompts

Tier-aware prompts

The system prompt varies by user role:

Standard (subscriber, student, professional):

> "You are an expert AI Analyst for the Health Transformation Review (HTR). Your audience consists of healthcare executives, policy makers, and economists. Answer questions thoroughly and professionally, citing specific policies, data, and source documents where relevant. When referencing a document, name it explicitly (e.g. 'According to the Wyman Report...' or 'Vermont Act 167 states...'). Focus on policy, economics, technology, clinical outcomes, and health equity."

Advisory / Admin:

Adds to the standard prompt:

> "You are speaking with an ADVISORY-tier client — a senior healthcare leader or organizational decision-maker. Provide deeper strategic analysis, quantitative benchmarking, and actionable recommendations tailored to organizational implementation. Feel free to draw on comparative case studies and multi-state examples."

Custom system prompt:

Clients can pass `systemPrompt` in the request body (max 800 characters). This completely overrides the tier-based prompts.

Conversation Memory

Each chat request reconstructs `ChatMemoryBuffer` from the `history` array sent with the request.

- Token limit: 4,096 tokens
- Role mapping: `"user"` → `MessageRole.USER`, `"ai"` → `MessageRole.ASSISTANT`
- Empty messages are skipped
- Each message is truncated to 4,000 characters in the request validator

The frontend maintains conversation history in browser `localStorage` (key: `htr-chat-history`) and sends the full history array with each request. The backend is stateless — it does not store conversation history.

Follow-up Question Suggestions

After each AI response, the frontend calls `POST /api/suggest` with the current conversation history.

The backend sends the last 6 conversation turns to the Groq LLM with this prompt:

```
Based on this health policy conversation, suggest exactly 3 concise follow-up
questions
the user might want to ask next. Return ONLY a JSON array of 3 strings, no
other text.

Conversation:
[last 6 messages truncated to 400 chars each]

Return format: ["question 1", "question 2", "question 3"]
```

The response is parsed as a JSON array and returned as `{ "suggestions": [...] }`.

No auth required. Returns `{ "suggestions": [] }` on error.

Authentication and Authorization

JWT verification (production)

The Python backend verifies Supabase JWTs:

```
payload = jwt.decode(
    token,
    SUPABASE_JWT_SECRET,
    algorithms=["HS256"],
    audience="authenticated",
)
```

`SUPABASE_JWT_SECRET` is found in: Supabase Dashboard → Settings → API → JWT Secret

Role check

After verifying the JWT, the backend fetches `user_roles` from Supabase:

```
res = supabase.table("user_roles").select("role").eq("user_id",
user_id).execute()
```

The highest role is determined by iterating through the hierarchy: `free → subscriber → student → professional → advisory → admin`.

If the role is below `subscriber`, a 403 is returned with:

```
{ "detail": "A Subscriber plan or higher is required to use the AI Analyst." }
```

Dev mode

If `SUPABASE_JWT_SECRET` is not set, the backend logs a warning and accepts all requests as `subscriber`. This allows local development without a real Supabase JWT.

The frontend proxy sends `Authorization: Bearer dev` as a fallback when no Authorization header is present.

Sanity Ingestion Details

API endpoint

```
GET https://fxz10xl7.api.sanity.io/v2023-10-01/data/query/production?query=...
Authorization: Bearer $SANITY_API_TOKEN
```

Retry logic

- 3 attempts per content type
- HTTP 429 (rate limit): exponential backoff starting at 1.5 seconds
- Other errors: retry with backoff, log final failure

Body text extraction

Sanity Portable Text is converted to plain text using the GROQ `pt::text()` function in the query itself. Example:

```
*[_type=="policyAnalysis" && defined(slug.current)]{
    _id, title, pillar, summary,
    "bodyText": pt::text(body)
}
```

Document construction

For each Sanity result, a LlamaIndex `Document` is built from concatenated fields:

1. `title` or `term`
2. `summary`, `description`, or `abstract`
3. `learningObjectives` (joined with ". ")
4. `bodyText`

Fields are joined with double newlines. Final text is truncated to 8,000 characters.

Adding New PDFs to the Knowledge Base

1. Place the PDF file in `backend/data/`
2. Trigger a re-index: `POST /api/ingest`
3. The PDF will be chunked by page and embedded automatically

No code changes required. All `.pdf` files in the directory are detected on every index build.

Adding New Sanity Content Types to the Knowledge Base

Edit `backend/main.py` → `SANITY_QUERIES` dict:

```
SANITY_QUERIES = {
    # ... existing queries ...
    "newType": """*[_type=="newType" && defined(slug.current)]{
        _id, title, pillar, summary,
        "bodyText": pt::text(body)
    }""",
}
```

Then trigger a re-index. The new type will be fetched and embedded on the next build.

Monitoring and Health

Health endpoint

```
curl https://your-backend.railway.app/health
```

Response:

```
{
  "status": "ok",
  "index_ready": true,
  "model": "llama-3.3-70b-versatile",
  "embedding_model": "text-embedding-3-small",
```

```
  "vector_store": "pgvector",
  "auth_enabled": true
}
```

- `index_ready: false` means the server started but the index build hasn't completed yet. Requests to `/api/chat` will return 503 until this is `true`.
- `vector_store: "local_json"` means `SUPABASE_DB_URL` is not configured and the fallback is in use.
- `auth_enabled: false` means `SUPABASE_JWT_SECRET` is not set — dev mode, all requests accepted.

Frontend backend status indicator

The `BackendStatus` component in the chat page polls the frontend's `/api/health` route, which proxies to the Python backend's `/health` endpoint. It shows a green dot when the backend is ready.

Railway health check

Railway pings `/health` every 30 seconds with a 10-second timeout. If this fails 3 times, Railway restarts the service.

Troubleshooting

"Index not ready — try again in a few seconds" (503)

The server started but the index build is still running. Wait 1–3 minutes. Check Railway logs for progress.

Responses don't reflect recent Sanity content

The index has not been rebuilt since the content was published. Run `POST /api/ingest`.

"A Subscriber plan or higher is required" (403)

In production: the user's JWT does not have a subscriber+ role. Check the `user_roles` table in Supabase.

In dev: if you see this, `SUPABASE_JWT_SECRET` is set on the backend but no valid JWT is being sent from the frontend. Either unset `SUPABASE_JWT_SECRET` for dev, or pass a valid JWT.

"Cannot reach the AI backend" (503)

The Python backend is not running. Locally: `cd backend && uvicorn main:app --reload --port 8000`. In production: check Railway deploy logs.

Index builds from Sanity but has stale data

Sanity API token may have expired or lost permissions. Check the token in Sanity's API settings. Tokens are project-scoped — ensure it has `read` access to the `production` dataset.

pgvector connection fails, falling back to local storage

`SUPABASE_DB_URL` is incorrect or the pgvector extension is not enabled. In Supabase: Database → Extensions → enable `vector`. The connection string must use the connection pooler URL (port 6543), not the direct connection (port 5432).

Cost Estimates

Service	Usage	Approximate Cost
OpenAI embeddings	Per index build (~1000 documents × ~1000 tokens avg)	~$0.02 per rebuild
Groq LLM	Per chat message	Very low (Groq is fast and cheap)

Service	Usage	Approximate Cost
Supabase	pgvector storage + DB queries	Covered by Supabase plan
Railway	Backend hosting	Depends on plan

OpenAI embedding cost is only incurred during index builds (when `/api/ingest` is called or on cold start without an existing index). Chat requests only use Groq (no OpenAI charges).